

CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Engineering and Technology

Devang Patel Institute of Advance Technology & Research

Department Of Computer Science & Engineering

A.Y. 2026-2027[Odd]

Lab Manual

CEUE203: Object Oriented Programming.

Name: Vaidehi Parmar

ID: 25DCS073

PRACTICAL 4

PART A:

1. Password strength (PasswordChecker.java + Driver.java)

(a) In PasswordChecker, check four rules — length ≥ 8, contains an uppercase letter, contains a digit, contains a special character — using regex (e.g. ".*[A-Z].*") or String methods.

(b) Write strength(String pw) that counts how many rules pass and returns "Weak" (0–1), "Medium" (2–3) or "Strong" (4).

(c) In Driver.main: test several passwords; for each, print which rules passed and the final label.

Expected output: "abc" → Weak; "Abcd1234!" → Strong.

Solution:

PasswordChecker.java

```
public class PasswordChecker {
    public static boolean lengthCheck(String pw) {
        return pw.length() >= 8;
    }

    public static boolean uppercaseCheck(String pw) {
        return pw.matches(regex: ".*[A-Z].*");
    }

    public static boolean digitCheck(String pw) {
        return pw.matches(regex: ".*\\d.*");
    }

    public static boolean specialCheck(String pw) {
        return pw.matches(regex: ".*[^a-zA-Z0-9].*");
    }

    public static String strength(String pw) {
        int count = 0;

        if (lengthCheck(pw)) count++;
        if (uppercaseCheck(pw)) count++;
        if (digitCheck(pw)) count++;
        if (specialCheck(pw)) count++;

        if (count <= 1) {
            return "Weak";
        } else if (count <= 3) {
            return "Medium";
        } else {
            return "Strong";
        }
    }
```

Driver4.java

```java
public class Driver4 {
    Run | Debug
    public static void main(String[] args) {
        String[] passwords = {
            "abc",
            "abcdefgh",
            "Abcdefgh",
            "Abcd1234",
            "Abcd1234!"
        };

        for (String pw : passwords) {
            System.out.println("Password: " + pw);
            System.out.println("Length >= 8: " + PasswordChecker.lengthCheck(pw));
            System.out.println("Uppercase: " + PasswordChecker.uppercaseCheck(pw));
            System.out.println("Digit: " + PasswordChecker.digitCheck(pw));
            System.out.println("Special character: " + PasswordChecker.specialCheck(pw));
            System.out.println("Strength: " + PasswordChecker.strength(pw));
            System.out.println();
        }
    }
}
```

Output:

```
Password: abc
Length >= 8: false
Uppercase: false
Digit: false
Special character: false
Strength: Weak

Password: abcdefgh
Length >= 8: true
Uppercase: false
Digit: false
Special character: false
Strength: Weak

Password: Abcdefgh
Length >= 8: true
Uppercase: true
Digit: false
Special character: false
Strength: Medium

Password: Abcd1234
Length >= 8: true
Uppercase: true
Digit: true
Special character: false
Strength: Medium

Password: Abcd1234!
Length >= 8: true
Uppercase: true
Digit: true
Special character: true
Strength: Strong
```

2. Chat-log filter (ChatFilter.java + Driver.java)

(a) In Driver.main, define a String[] of log lines like “10:05 alice Hello there” (include one malformed line with fewer than 3 parts).

(b) Read a keyword from the user.

(c) For each line, split into time/user/message with split(“ ”, 3); skip lines that do not have 3 parts.

(d) If the message contains the keyword (case-insensitive), increment a counter and append “time user: message” to a StringBuilder.

(e) Print “Matches: N” and the built report.

Expected output: keyword “hello” over 3 lines (1 malformed) → “Matches: 1” and the matching line.

Solution:

ChatFilter.java

```java
public class ChatFilter {
    public static boolean containsKeyword(String message, String keyword) {
        return message.toLowerCase().contains(keyword.toLowerCase());
    }
}
```

Driver5.java

```java
import java.util.Scanner;

public class Driver5 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String[] logs = {
            "10:05 alice Hello there",
            "10:10 bob How are you?",
            "10:15 charlie",
            "10:20 david Good morning"
        };

        System.out.print("Enter keyword: ");
        String keyword = sc.nextLine();

        int count = 0;
        StringBuilder report = new StringBuilder();

        for (String line : logs) {
            String[] parts = line.split(" ", 3);

            if (parts.length < 3) {
                continue;
            }

            String time = parts[0];
            String user = parts[1];
            String message = parts[2];

            if (ChatFilter.containsKeyword(message, keyword)) {
                count++;
                report.append(time)
                      .append(" ")
                      .append(user)
                      .append(": ")
                      .append(message)
                      .append("\n");
            }
        }

        System.out.println("Matches: " + count);
        System.out.print(report);

        sc.close();
    }
}
```

Output:

```
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> javac Driver5.java
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> java Driver5
Enter keyword: hello
Matches: 1
10:05 alice: Hello there
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> javac Driver5.java
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> java Driver5
Enter keyword: good
Matches: 1
10:20 david: Good morning
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> javac Driver5.java
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> java Driver5
Enter keyword: xyz
Matches: 0
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3>
```

3. Template filler (TemplateFiller.java + Driver.java)

(a) In Driver.main, define a template like “Dear {name}, order {id} ships {date}.” and two parallel arrays: names {“name”, “id”} and values {“Riya”, “A07”} (note {date} is not supplied).

(b) Compile a Pattern for \{(\w+)\} and get a Matcher on the template.

(c) Walk the matches with a StringBuilder: for each {placeholder}, look up its value in the names array and replace with the value, or with “[?]” if not found.

(d) Print the filled template.

Expected output: the unsupplied {date} becomes [?].

Solution:

TemplateFiller.java

```java
import java.util.regex.Matcher;
import java.util.regex.Pattern;
public class TemplateFiller {
    public static String fill(String template, String[] names, String[] values) {
        Pattern pattern = Pattern.compile(regex: "\\{(\\w+)\\}");
        Matcher matcher = pattern.matcher(template);
        StringBuilder result = new StringBuilder();
        int lastEnd = 0;

        while (matcher.find()) {
            result.append(template, lastEnd, matcher.start());

            String placeholder = matcher.group(group: 1);
            String replacement = "[?]";

            for (int i = 0; i < names.length; i++) {
                if (names[i].equals(placeholder)) {
                    replacement = values[i];
                    break;
                }
            }

            result.append(replacement);
            lastEnd = matcher.end();
        }
        result.append(template, lastEnd, template.length());
        return result.toString();
    }
}
```

Driver6.java

```java
public class Driver6 {
    Run | Debug
    public static void main(String[] args) {
        String template = "Dear {name}, order {id} ships {date}.";

        String[] names = {"name", "id"};
        String[] values = {"Riya", "A07"};

        String result = TemplateFiller.fill(template, names, values);

        System.out.println(result);
    }
}
```

Output:

```
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> javac Driver6.java
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> java Driver6
Dear Riya, order A07 ships [?].
```

Part B — Project Work (Hour 2): MiniBank

Your classes exist, but the bank must never trust raw user input. In this lab you build MiniBank's validation and command-parsing layer — regex checks for mobile, email, PAN and IFSC, and a parser that turns a typed line such as "DEPOSIT AC0001 500" into a Command object. Practical 14 will run every menu action through this layer.

1. Create a class named Validator with public static methods that use compiled regular expressions (Pattern): isValidMobile(String), isValidEmail(String), isValidPan(String) and isValidIfsc(String); each returns a boolean.

2. Define an enum named TransactionType with the constants DEPOSIT, WITHDRAW and TRANSFER. Then define a record named Command with three fields: a TransactionType type, a String accountNumber and a long amount.

3. Create a class named CommandParser with a public static method parse(String line) that splits a line such as "DEPOSIT AC0001 500" with split(), converts the parts, and returns a Command object.

4. Create a class named StatementFormatter with a public static method buildStatement(Account account) that uses a StringBuilder to assemble a multi-line account statement string.

5. In main, test each validator with one correct and one wrong input, then parse a sample command and print its three parts.

Solution:

```
import java.util.Scanner;
import java.util.regex.Pattern;
import java.util.Objects;

public class MiniBank {

    public static void main(String[] args) {
        BankInfo bankInfo = new BankInfo("MiniBank", "Anand Branch");
        System.out.println("==================================");
        System.out.println(bankInfo);
        System.out.println("==================================");

        Account[] accounts = {
            new Account("Vaidehi", 1000),
            new Account("Riya"),
            new Account("Aarav", 5000)
        };

        accounts[0].deposit(2000);
        accounts[0].withdraw(500);
        accounts[1].deposit(3000);
        accounts[1].withdraw(1000);
        accounts[2].deposit(1500);
        accounts[2].withdraw(2000);

        System.out.println("\nAccount Details:");
```

```
for (Account account : accounts) {
    System.out.println(account);
}

System.out.println("\nAccount Comparison:");
System.out.println(accounts[0].equals(accounts[1]));
System.out.println(accounts[0] instanceof Account);

System.out.println("\nValidators:");
System.out.println("Mobile valid: " + Validator.isValidMobile("9876543210"));
System.out.println("Mobile invalid: " + Validator.isValidMobile("1234567890"));

System.out.println("Email valid: " + Validator.isValidEmail("user@gmail.com"));
System.out.println("Email invalid: " + Validator.isValidEmail("user@gmail"));

System.out.println("PAN valid: " + Validator.isValidPan("ABCDE1234F"));
System.out.println("PAN invalid: " + Validator.isValidPan("ABC1234F"));

System.out.println("IFSC valid: " + Validator.isValidIfsc("SBIN0001234"));
System.out.println("IFSC invalid: " + Validator.isValidIfsc("SBI1234"));

System.out.println("Amount valid: " + Validator.isValidAmount("500"));
System.out.println("Amount invalid: " + Validator.isValidAmount("-500"));

Command command = CommandParser.parse("DEPOSIT AC0001 500");

System.out.println("\nParsed Command:");
System.out.println("Type: " + command.type());
System.out.println("Account Number: " + command.accountNumber());
```

```
        System.out.println("Amount: " + command.amount());

        System.out.println("\nStatement:");
        System.out.println(StatementFormatter.buildStatement(accounts[0]));

        Scanner scanner = new Scanner(System.in);

        while (true) {
            System.out.println("\n===== MiniBank Menu =====");
            System.out.println("1. Open Account");
            System.out.println("2. Deposit");
            System.out.println("3. Withdraw");
            System.out.println("4. Transfer");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");

            int choice = scanner.nextInt();

            MenuOption option = switch (choice) {
                case 1 -> MenuOption.OPEN_ACCOUNT;
                case 2 -> MenuOption.DEPOSIT;
                case 3 -> MenuOption.WITHDRAW;
                case 4 -> MenuOption.TRANSFER;
                case 5 -> MenuOption.EXIT;
                default -> null;
            };

            if (option == null) {
                System.out.println("Invalid choice.");
```

```
                continue;
            }

            String message = switch (option) {
                case OPEN_ACCOUNT -> "Open Account — to be implemented in a later lab";
                case DEPOSIT -> "Deposit — to be implemented in a later lab";
                case WITHDRAW -> "Withdraw — to be implemented in a later lab";
                case TRANSFER -> "Transfer — to be implemented in a later lab";
                case EXIT -> "Exiting MiniBank...";
            };

            System.out.println(message);

            if (option == MenuOption.EXIT) {
                break;
            }
        }

        System.out.println("Thank you for using MiniBank. Goodbye!");
        scanner.close();
    }
}

record BankInfo(String name, String branch) {
}

enum MenuOption {
    OPEN_ACCOUNT,
    DEPOSIT,
```

```
    WITHDRAW,
    TRANSFER,
    EXIT
}

class Customer implements Cloneable {
    private String name;
    private String email;
    private String mobile;
    private final String customerId;
    private static long customerCounter = 100;

    private Address address;

    public Customer(String name, String email, String mobile) {
        this.name = name;
        this.email = email;
        this.mobile = mobile;
        this.customerId = generateCustomerId();
    }

    private static String generateCustomerId() {
        return "CUST" + (++customerCounter);
    }

    public String getName() {
        return name;
    }
```

```
    public String getEmail() {
        return email;
    }

    public String getMobile() {
        return mobile;
    }

    public String getCustomerId() {
        return customerId;
    }

    public Address getAddress() {
        return address;
    }

    public void setAddress(Address address) {
        this.address = address;
    }

    public Customer clone() {
        try {
            return (Customer) super.clone();
        } catch (CloneNotSupportedException e) {
            throw new AssertionError();
        }
    }

    public static class Address {
```

```
        private String line;
        private String city;
        private String pincode;

        public Address(String line, String city, String pincode) {
            this.line = line;
            this.city = city;
            this.pincode = pincode;
        }

        public String getLine() {
            return line;
        }

        public String getCity() {
            return city;
        }

        public String getPincode() {
            return pincode;
        }
    }
}

class Account {
    private final String accountNumber;
    private String ownerName;
    private long balance;
    private boolean active;
```

```
    private static long accountCounter = 0;

    public Account(String ownerName, long openingBalance) {
        this.accountNumber = String.format("AC%04d", ++accountCounter);
        this.ownerName = ownerName;
        this.balance = openingBalance;
        this.active = true;
    }

    public Account(String ownerName) {
        this(ownerName, 0);
    }

    public void deposit(long amount) {
        balance += amount;
    }

    public boolean withdraw(long amount) {
        if (balance >= amount) {
            balance -= amount;
            return true;
        }
        return false;
    }

    public String getAccountNumber() {
        return accountNumber;
    }
```

```
public String getOwnerName() {
    return ownerName;
}

public long getBalance() {
    return balance;
}

public boolean isActive() {
    return active;
}

@Override
public String toString() {
    return "Account Number: " + accountNumber +
           ", Owner: " + ownerName +
           ", Balance: " + balance;
}

@Override
public boolean equals(Object o) {
    if (this == o) {
        return true;
    }

    if (!(o instanceof Account)) {
        return false;
    }
```

```
        Account account = (Account) o;
        return accountNumber.equals(account.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}

class Validator {
    private static final Pattern MOBILE =
            Pattern.compile("^[6-9][0-9]{9}$");

    private static final Pattern EMAIL =
            Pattern.compile("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");

    private static final Pattern PAN =
            Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");

    private static final Pattern IFSC =
            Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");

    private static final Pattern AMOUNT =
            Pattern.compile("^[1-9][0-9]*$");

    public static boolean isValidMobile(String mobile) {
        return mobile != null && MOBILE.matcher(mobile).matches();
```

```
    }

    public static boolean isValidEmail(String email) {
        return email != null && EMAIL.matcher(email).matches();
    }

    public static boolean isValidPan(String pan) {
        return pan != null && PAN.matcher(pan).matches();
    }

    public static boolean isValidIfsc(String ifsc) {
        return ifsc != null && IFSC.matcher(ifsc).matches();
    }

    public static boolean isValidAmount(String amount) {
        return amount != null && AMOUNT.matcher(amount).matches();
    }
}

enum TransactionType {
    DEPOSIT,
    WITHDRAW,
    TRANSFER
}

record Command(TransactionType type, String accountNumber, long amount) {
}

class CommandParser {
```

```
    public static Command parse(String line) {
        if (line == null || line.trim().isEmpty()) {
            throw new IllegalArgumentException("Invalid command");
        }

        String[] parts = line.trim().split("\\s+");

        if (parts.length != 3) {
            throw new IllegalArgumentException(
                    "Error: command must contain exactly 3 parts"
            );
        }

        TransactionType type;

        try {
            type = TransactionType.valueOf(parts[0].toUpperCase());
        } catch (IllegalArgumentException e) {
            throw new IllegalArgumentException("Error: invalid transaction type");
        }

        if (!Validator.isValidAmount(parts[2])) {
            throw new IllegalArgumentException("Error: invalid amount");
        }

        long amount;

        try {
            amount = Long.parseLong(parts[2]);
```

```
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("Error: amount is too large");
        }

        return new Command(type, parts[1], amount);
    }
}

class StatementFormatter {
    public static String buildStatement(Account account) {
        StringBuilder statement = new StringBuilder();

        statement.append("========== Account Statement ==========\n");
        statement.append("Account Number: ")
                .append(account.getAccountNumber())
                .append("\n");
        statement.append("Owner Name: ")
                .append(account.getOwnerName())
                .append("\n");
        statement.append("Balance: ")
                .append(account.getBalance())
                .append("\n");
        statement.append("Status: ")
                .append(account.isActive() ? "Active" : "Inactive")
                .append("\n");
        statement.append("=======================================\n");

        return statement.toString();
    }
```

}

Output:

```
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> javac MiniBank.java
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> java MiniBank
================================
BankInfo[name=MiniBank, branch=Anand Branch]
================================

Account Details:
Account Number: AC0001, Owner: Vaidehi, Balance: 2500
Account Number: AC0002, Owner: Riya, Balance: 2000
Account Number: AC0003, Owner: Aarav, Balance: 4500

Account Comparison:
false
true

Validators:
Mobile valid: true
Mobile invalid: false
Email valid: true
Email invalid: false
PAN valid: true
PAN invalid: false
IFSC valid: true
IFSC invalid: false
Amount valid: true
Amount invalid: false

Parsed Command:
Type: DEPOSIT
Account Number: AC0001
Amount: 500

Statement:
========== Account Statement ==========
Account Number: AC0001
Owner Name: Vaidehi
Balance: 2500
Status: Active
=======================================


===== MiniBank Menu =====
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 4
Transfer ? to be implemented in a later lab

===== MiniBank Menu =====
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 5
Exiting MiniBank...
Thank you for using MiniBank. Goodbye!
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3>
```